

INTERNSHIP PROJECT REPORT

on

DocuMind RAG Assistant

Grounded Multi-Format Document Intelligence Assistant

Submitted in partial fulfilment of the requirements of the

SHADOWFOX Internship Program

Domain: AI Engineer

Submitted by

ILAMARAN R

B.Tech - Information Technology, 3rd Year

PKIET

Internship Duration

August 1, 2026 - September 2, 2026

Declaration

I, ILAMARAN R, a 3rd Year B.Tech Information Technology student at PKIET, hereby declare that the internship project report entitled “DocuMind RAG Assistant – Grounded Multi-Format Document Intelligence Assistant” is an authentic record of the work carried out by me during my internship with SHADOWFOX under the AI Engineer domain, from August 1, 2026 to September 2, 2026.

The system architecture, source code, and functional results presented in this report are based on the project developed by me during this internship period. This work has not been submitted elsewhere for the award of any other degree or diploma.

ILAMARAN R

B.Tech Information Technology, 3rd Year
PKIET

Acknowledgement

I would like to express my sincere gratitude to SHADOWFOX for providing me the opportunity to undertake this internship in the AI Engineer domain. This experience has given me valuable practical exposure to real-world AI engineering practices, including retrieval-augmented generation, vector search systems, and production-style application design. Finally, I would like to thank my family and mentor for their constant support and motivation.

ILAMARAN R

Table of Contents

Declaration.....	2
Acknowledgement.....	3
Abstract.....	5
1. Introduction.....	6
2. Problem Statement.....	6
3. Objectives.....	6
4. Technologies Used.....	8
5. System Requirements.....	8
6. System Architecture / Workflow.....	10
7. Project Features and Modules.....	12
8. Implementation / Methodology.....	13
9. User Interface / Screenshots.....	14
10. Testing and Results.....	18
11. Advantages.....	19
12. Limitations.....	19
13. Future Enhancements.....	19
14. Conclusion.....	20
15. References.....	20

Abstract

Large Language Models (LLMs) are powerful general-purpose reasoning engines, but they are fundamentally limited by fixed knowledge cutoffs and a tendency to hallucinate when asked about proprietary, private, or recently created documents. DocuMind RAG Assistant is a production-style Retrieval-Augmented Generation (RAG) system built to address this limitation. It allows a user to upload multi-format documents (PDF, TXT, and Markdown), automatically parses and chunks the content, generates dense vector embeddings locally using SentenceTransformers, and indexes them in a FAISS vector store for fast similarity search.

When a user asks a question, the system retrieves the most relevant chunks, filters them using a score-threshold reranker, and passes them to the Google Gemini LLM under a strict grounding prompt that forbids the use of outside knowledge. A custom groundedness validator then cross-checks the generated answer's key terms against the retrieved context and assigns a confidence level (HIGH / MEDIUM / LOW), explicitly detecting refusals when no supporting evidence exists. Every answer is returned with granular, verifiable citations — filename, page number, chunk ID, and similarity score.

The entire query lifecycle is orchestrated using LangGraph as a five-node state machine, and the system is exposed through a decoupled FastAPI backend and a Streamlit-based web interface, containerized using Docker for reproducible deployment. This report documents the problem this project solves, its architecture, implementation details, and the functional results observed during development and testing.

1. Introduction

The SHADOWFOX Internship Program is structured across three progressive levels — Beginner, Intermediate, and Advanced — each designed to assess a growing depth of AI engineering capability. The Advanced Level task, as specified in the internship's task list document, requires the intern to build and explain the architecture of a full AI system: justifying why each component was chosen, how retrieval quality is handled, how hallucinations are reduced, and how the project reflects practical AI engineering beyond a simple demo application.

In response to this brief, this project implements DocuMind RAG Assistant, a Retrieval-Augmented Generation (RAG) system for grounded, citation-backed question answering over user-uploaded documents. Unlike a basic chatbot wrapper around an LLM API, this system is engineered as a multi-layer pipeline: document ingestion and chunking, local embedding generation, vector similarity search, relevance-based reranking, constrained LLM generation, and an explicit post-hoc groundedness validation stage.

This report documents the problem being solved, the system's design and architecture, the technologies used, the implementation methodology, and the functional results observed while running the application locally.

2. Problem Statement

General-purpose LLMs cannot answer questions about documents they were never trained on — internal reports, personal notes, or newly created files. When forced to answer regardless, they frequently hallucinate: producing fluent but factually unsupported text. Naive RAG prototypes only partially solve this, since they typically retrieve context but still allow the LLM to answer freely, without any check on whether the final answer is actually supported by what was retrieved, and without transparent, verifiable citations that let a user confirm the source of a claim.

The problem this project addresses is therefore twofold: (1) how to retrieve the most relevant pieces of a user's own documents for a given question, and (2) how to constrain and verify that the generated answer stays strictly grounded in that retrieved evidence — refusing to answer, rather than guessing, when the evidence is insufficient.

3. Objectives

- Design a multi-format document ingestion pipeline supporting PDF, TXT, and Markdown files with page-level metadata tracking.
- Implement local, cost-free vector embeddings using SentenceTransformers and a persistent FAISS vector index for semantic search.
- Build a relevance-based reranking stage to filter out low-similarity, noisy retrieval results before they reach the LLM.
- Integrate a large language model (Google Gemini) under a strict, context-only grounding prompt to minimise hallucination.
- Implement an explicit, independent groundedness validation layer that scores answer–context overlap and detects grounded refusals.
- Provide granular, verifiable source citations (filename, page number, chunk ID, similarity score) for every generated answer.
- Orchestrate the end-to-end query lifecycle as a reproducible, inspectable LangGraph state machine.
- Expose the system via a decoupled FastAPI backend and a Streamlit front-end, and containerize both using Docker.

4. Technologies Used

The project uses a modern, production-oriented Python stack. Each technology was selected for a specific architectural reason rather than convenience, as summarised below.

Layer	Technology	Purpose
Programming Language	Python 3.10+	Core application language
Backend API	FastAPI, Uvicorn, Pydantic v2	REST microservice, request validation, ASGI server
Frontend UI	Streamlit	Interactive chat-based web interface
Vector Database	FAISS (faiss-cpu)	Local vector similarity search (cosine similarity)
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Local, zero-cost dense text embeddings
LLM Engine	Google Gemini API (google-genai SDK)	Grounded natural-language answer generation
Orchestration	LangGraph, LangChain Core	Stateful multi-step RAG pipeline workflow
Document Parsing	pypdf, markdown	PDF text/page extraction, Markdown parsing
Containerization	Docker, docker-compose	Reproducible backend + frontend deployment
Testing	pytest, httpx	Unit and API integration testing

5. System Requirements

5.1 Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Language Runtime	Python 3.10 or higher
Package Manager	pip
Containerization (optional)	Docker & Docker Compose
Code Editor	VS Code (or equivalent)
External Service	Google Gemini API key (aistudio.google.com)

5.2 Hardware Requirements

Hardware	Requirement
Processor	Any modern multi-core CPU (GPU not required)
RAM	Minimum 4 GB (8 GB recommended for smooth embedding + LLM calls)
Storage	Sufficient disk space for uploaded documents and the FAISS index
Network	Internet connectivity required for Gemini API calls

Because embeddings are generated locally on CPU using a lightweight MiniLM model, the system does not require a GPU, keeping the hardware footprint modest and making the application easy to run on a standard laptop.

6. System Architecture / Workflow

The system follows a clean, decoupled, layered architecture. The Streamlit web UI never talks directly to the model or vector store — it communicates exclusively over HTTP with the FastAPI backend's REST API (/api/v1), which keeps the presentation layer and the AI engineering logic fully independent and separately testable/deployable.

Fig. System Architecture - DocuMind RAG Assistant

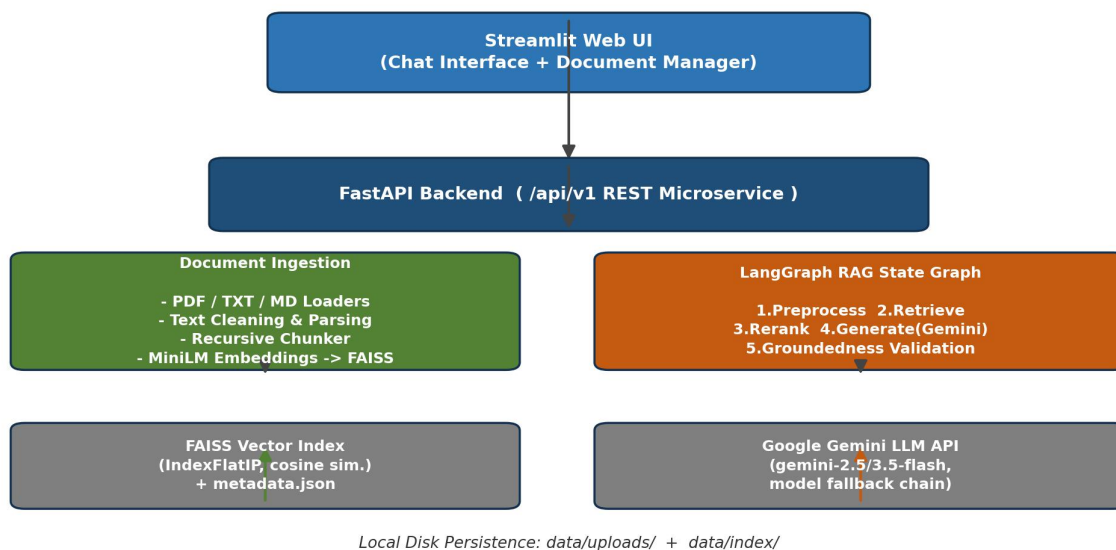


Fig 6.1: High-level system architecture of DocuMind RAG Assistant

6.1 Ingestion Flow

When a document is uploaded, it passes through: (1) a format-specific Document Loader (PDF via pypdf, TXT, or Markdown) that extracts raw text with page numbers; (2) a Document Parser that cleans and normalises the text; (3) a Recursive Text Chunker that splits the content into overlapping chunks (default chunk size 500 characters, overlap 100 characters) to preserve context across chunk boundaries; (4) the Embedding Service, which converts each chunk into a 384-dimensional normalised vector using the all-MiniLM-L6-v2 model; and (5) the FAISS Vector Store, which adds these vectors to a persistent IndexFlatIP index and writes the index and chunk metadata to disk (data/index/).

6.2 Query (RAG) Flow — LangGraph State Machine

Every user question is processed through a five-node LangGraph state machine, giving the pipeline an explicit, inspectable, and independently loggable structure rather than a single monolithic function:

Fig. LangGraph RAG Pipeline (5-Node State Machine)

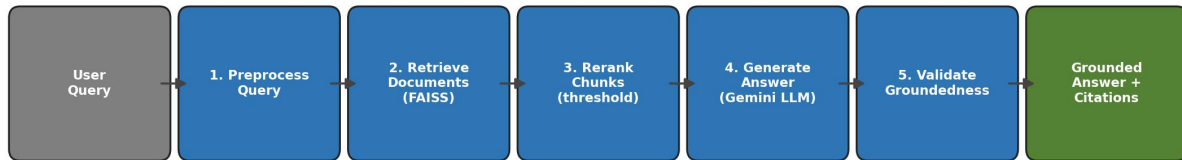


Fig 6.2: LangGraph 5-node RAG query pipeline

1. Preprocess Query — trims and sanitises the raw user input.
2. Retrieve Documents — embeds the query and performs a cosine-similarity search against the FAISS index, returning the top-K nearest chunks (K is configurable, default 5).
3. Rerank Chunks — discards any chunk below a similarity threshold (default 0.25) and removes duplicate text snippets, keeping only the highest-relevance, non-redundant evidence.
4. Generate Answer — the surviving chunks are formatted into a structured context block and passed to the Gemini LLM under a strict prompt that forbids answering outside the given context and instructs an explicit refusal phrase when evidence is insufficient.
5. Validate Groundedness — an independent validator (not the LLM itself) checks how many of the answer's content words actually appear in the retrieved context and assigns a confidence level.

7. Project Features and Modules

7.1 Key Features

- Multi-format ingestion: PDF (with page tracking), TXT, and Markdown.
- Local, zero-cost CPU vector embeddings (no external embedding API or per-call cost).
- Persistent FAISS vector index with add, delete-by-document, and clear-all operations.
- Score-threshold reranking to remove irrelevant / duplicate retrieval noise before generation.
- Anti-hallucination guardrail: a dedicated groundedness validator independent of the LLM's own claims.
- Granular source citations — filename, page number, chunk ID, similarity score, and text excerpt — shown in an accordion UI.
- Configurable retrieval settings (Top-K chunks, document scope filter) exposed directly in the UI.
- Resilient LLM calling with a model fallback chain, so generation degrades gracefully if a specific Gemini model is unavailable.
- Fully decoupled REST backend (FastAPI) and web frontend (Streamlit).
- Dockerized backend and frontend with a single docker-compose entry point.

7.2 Module Breakdown (Backend: app/)

Module	Responsibility
app/ingestion/	Document loaders, parser, and recursive text chunker
app/embeddings/	SentenceTransformer-based embedding service (singleton, lazy-loaded model)
app/retrieval/	FAISS vector store, retriever, and score-threshold reranker
app/generation/	Gemini LLM client, grounded prompt construction, model fallback logic
app/validation/	Groundedness validator (term-overlap scoring & refusal detection)
app/pipeline/	LangGraph RAG state machine wiring all five nodes together
app/api/	FastAPI routes and Pydantic request/response schemas
streamlit_app/	Chat UI, document manager sidebar, and citation display components

8. Implementation / Methodology

8.1 Vector Storage and Similarity Search

The FAISS vector store uses an IndexFlatIP (Inner Product) index. Because all embeddings are L2-normalised before being added, the inner product between two vectors is mathematically equivalent to their cosine similarity, giving a similarity score in the $[0, 1]$ range for every retrieved chunk. Document deletion is handled by reconstructing a fresh index from the retained vectors (via `index.reconstruct`), since FAISS's flat index does not support in-place removal.

8.2 Relevance Filtering (Reranking)

Rather than relying purely on the top-K nearest neighbours, the reranker module discards any retrieved chunk whose similarity score falls below a configurable threshold (0.25 by default) and removes chunks with near-duplicate text, so that only genuinely relevant, non-redundant evidence is passed on to the LLM. This step directly reduces the chance of the model being distracted by weakly related content.

8.3 Grounded Prompting

The prompt sent to Gemini explicitly instructs the model to answer only from the supplied context, to avoid outside knowledge or extrapolation, and to reply with a fixed refusal sentence (“Information not found in the provided documents.”) when the context is insufficient. Each context snippet is tagged with its source filename, page number, and chunk ID so the model can reference them directly.

8.4 Independent Groundedness Validation

Trusting an LLM's own claim that it “used the context” is not sufficient, so this system implements a second, independent verification step. The validator extracts content words (length > 3 , common stop-words removed) from the generated answer and checks what fraction of them also appear in the retrieved context text. Based on this overlap score, it assigns:

- HIGH confidence — overlap score ≥ 0.55
- MEDIUM confidence — overlap score ≥ 0.30 and < 0.55
- LOW confidence — overlap score < 0.30 (flagged as potentially ungrounded)

The validator also separately detects explicit refusal phrases (e.g. “information not found”) and marks these as a valid “grounded refusal” rather than a low-confidence answer, since correctly declining to answer is itself the desired, grounded behaviour.

8.5 API Design

The FastAPI backend exposes seven REST endpoints under `/api/v1`: `health`, `upload`, `ingest`, `query`, `documents (list)`, `documents/{filename} (delete)`, and `clear-index`. This separation allows the upload and ingestion steps to be triggered independently, and allows the same backend to be consumed by the Streamlit UI, automated tests, or any other client.

9. User Interface / Screenshots

The following screenshots were captured while running the application locally and demonstrate the working end-to-end system — from the Streamlit chat interface down to the FastAPI backend's internal pipeline logs.

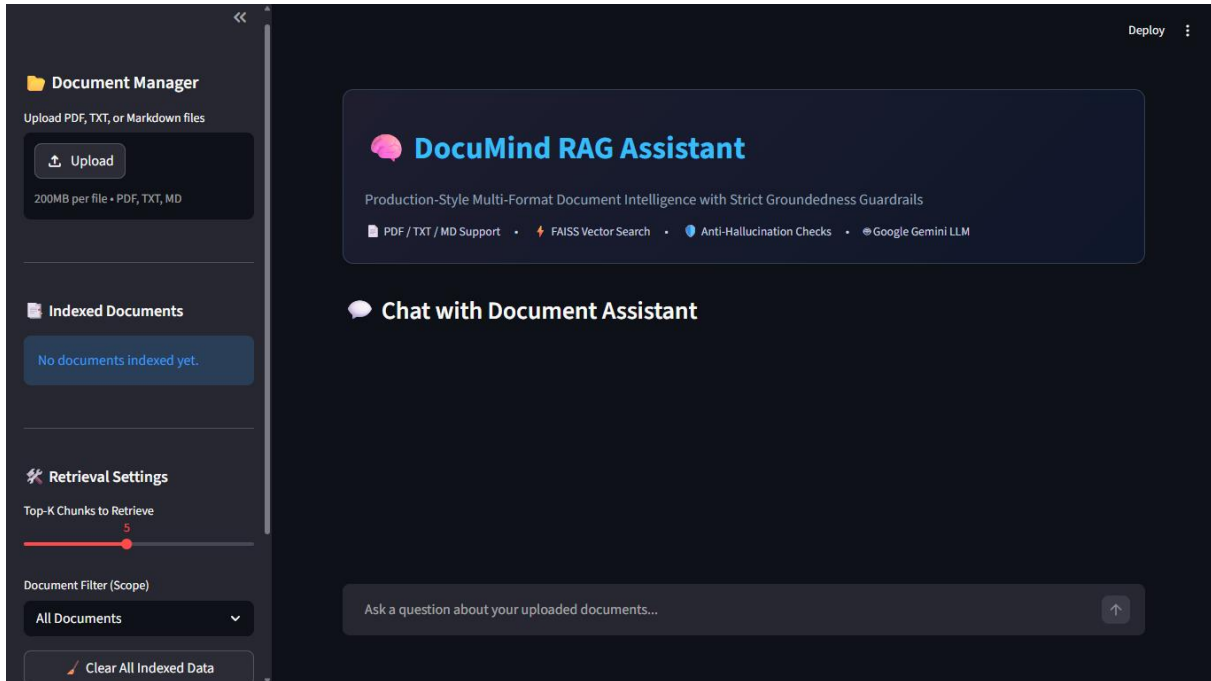


Fig 9.1: DocuMind RAG Assistant — initial interface before any document is indexed

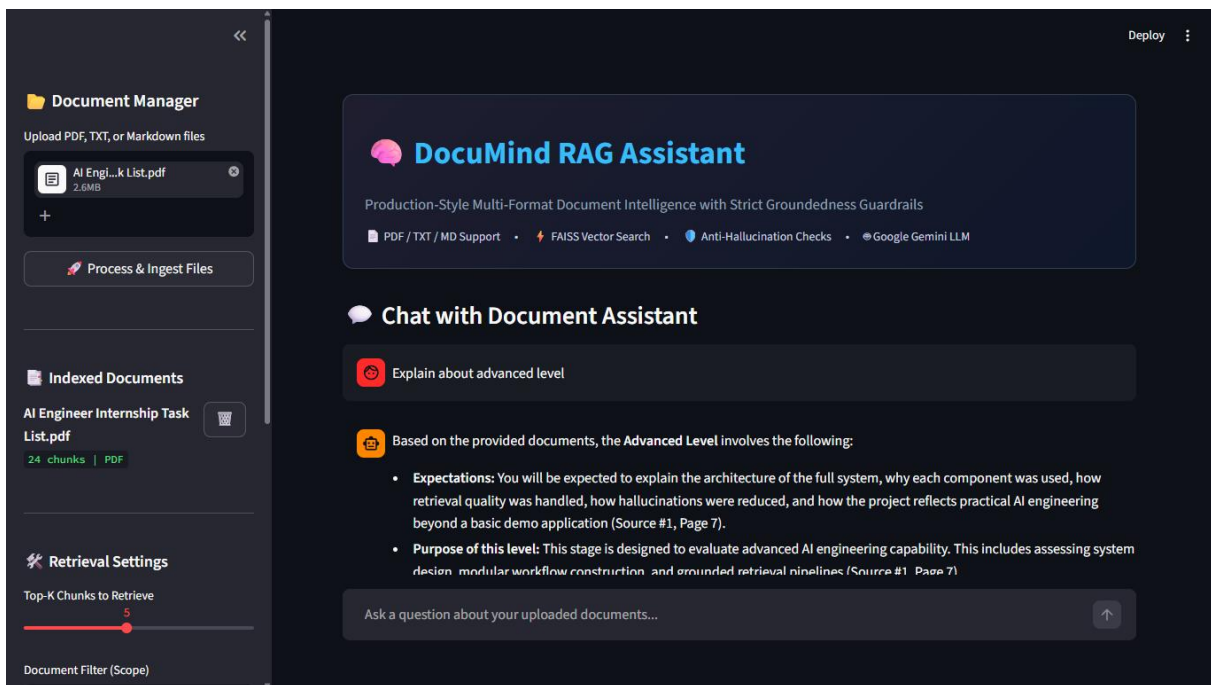


Fig 9.2: Application header and a sample grounded response after indexing “AI Engineer Internship Task List.pdf”

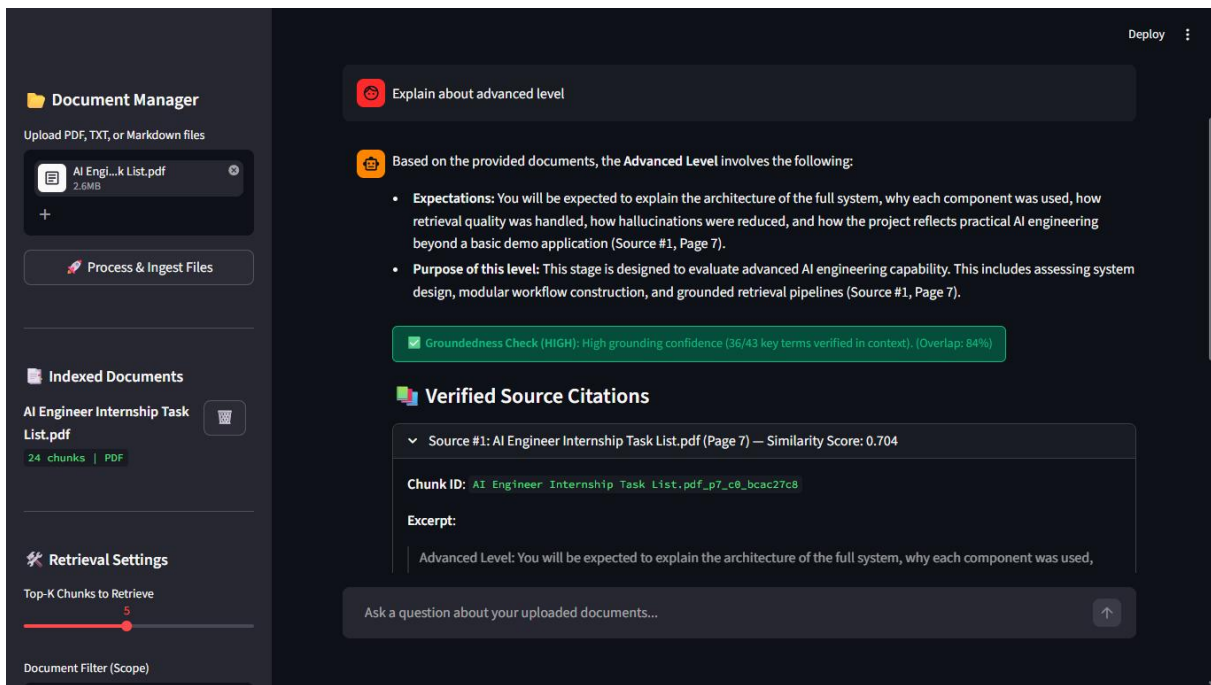


Fig 9.3: Query — “Explain about advanced level” — with HIGH groundedness confidence badge (36/43 key terms verified, 84% overlap)

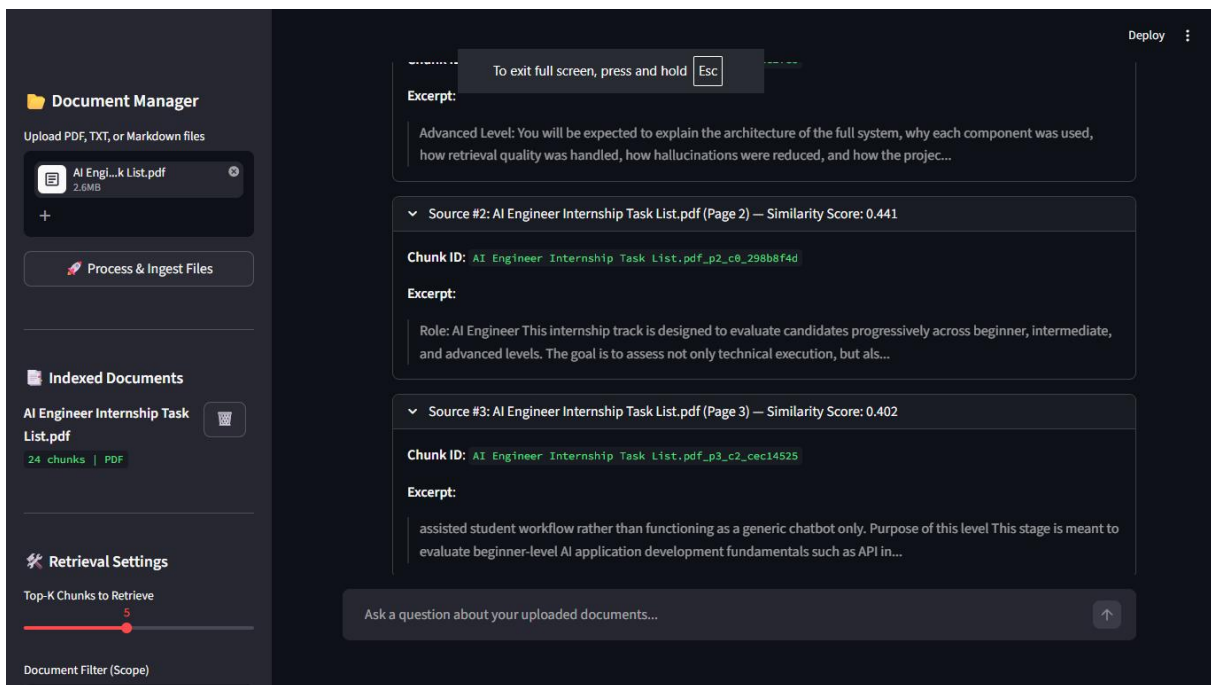


Fig 9.4: Verified source citations panel showing Source #1–#3 with filename, page number, chunk ID, and similarity score

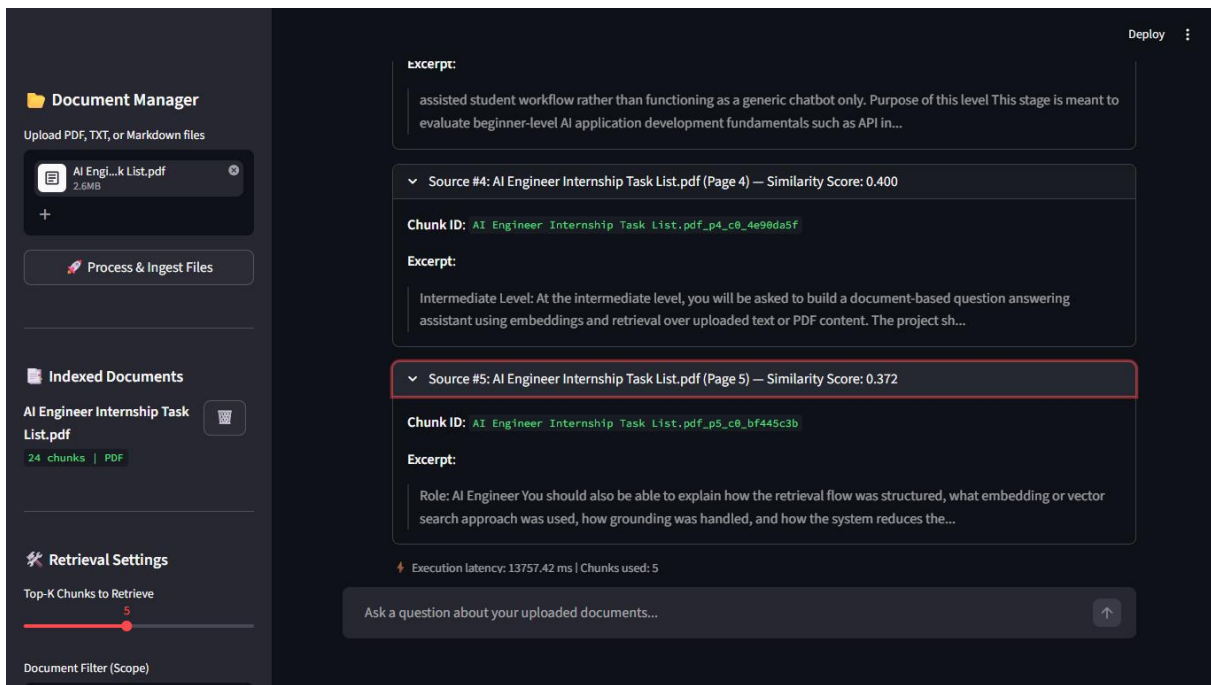


Fig 9.5: Remaining citations (Source #4, #5) and execution metrics — 13,757 ms latency, 5 chunks used

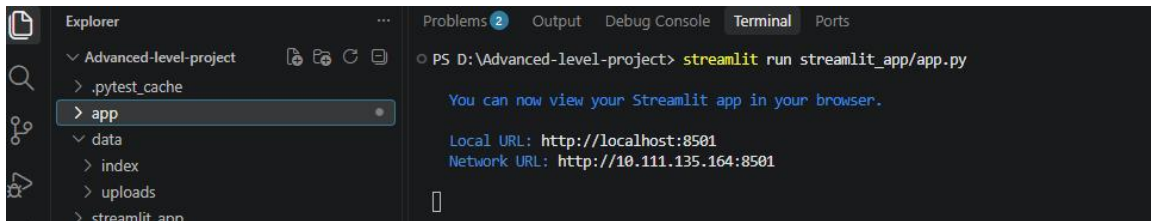


Fig 9.6: Streamlit frontend launched locally (streamlit run streamlit_app/app.py) on http://localhost:8501

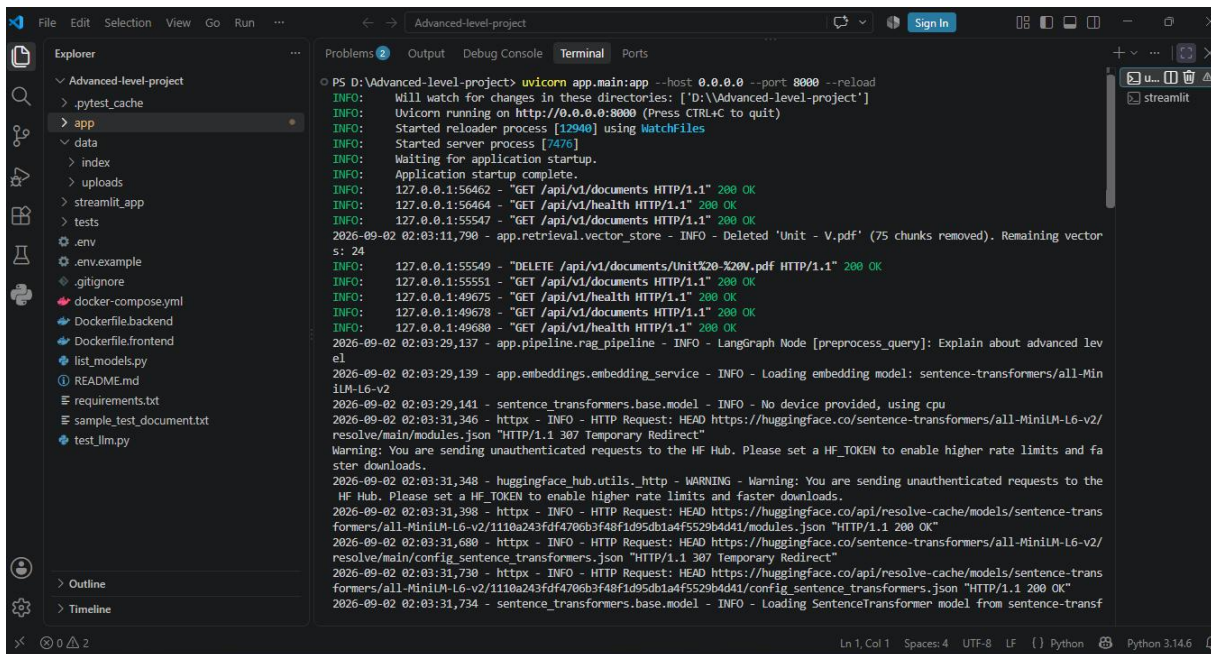


Fig 9.7: FastAPI backend (Uvicorn) live log — document deletion and index update (24 chunks removed)

```

P5 D:\Advanced-level-project> uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
formers/all-MiniLM-L6-v2/1118a243fdf4786b3f48f1d95db1a4f5529b4d41/config.json "HTTP/1.1 200 OK"
2026-09-02 02:03:37,112 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/
resolve/main/tokenizer_config.json "HTTP/1.1 307 Temporary Redirect"
2026-09-02 02:03:37,161 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentence-trans
formers/all-MiniLM-L6-v2/1118a243fdf4786b3f48f1d95db1a4f5529b4d41/tokenizer_config.json "HTTP/1.1 200 OK"
2026-09-02 02:03:37,588 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/sentence-transformers/all-Min
iLM-L6-v2/tree/main/additional_chat_templates?recursive=false&expand=false "HTTP/1.1 404 Not Found"
2026-09-02 02:03:37,890 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/sentence-transformers/all-Min
iLM-L6-v2/tree/main?recursive=true&expand=false "HTTP/1.1 200 OK"
2026-09-02 02:03:38,278 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/
resolve/main/1_Pooling/config.json "HTTP/1.1 307 Temporary Redirect"
2026-09-02 02:03:38,331 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/api/resolve-cache/models/sentence-trans
formers/all-MiniLM-L6-v2/1118a243fdf4786b3f48f1d95db1a4f5529b4d41/1_Pooling2Fconfig.json "HTTP/1.1 200 OK"
2026-09-02 02:03:38,707 - httpx - INFO - HTTP Request: HEAD https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/
resolve/main/2_Normalize/config.json "HTTP/1.1 404 Not Found"
2026-09-02 02:03:39,026 - httpx - INFO - HTTP Request: GET https://huggingface.co/api/models/sentence-transformers/all-Min
iLM-L6-v2 "HTTP/1.1 200 OK"
Batches: 100% | 1/1 [00:00<00:00, 2.86it/s]
2026-09-02 02:03:39,437 - app.pipeline.rag_pipeline - INFO - LangGraph Node [retrieve_documents]: Retrieved 5 raw chunks.
2026-09-02 02:03:39,442 - app.retrieval.reranker - INFO - Reranker: Kept 5 chunks, discarded 0 low-relevance chunks (Thres
hold=0.25)
2026-09-02 02:03:39,443 - app.pipeline.rag_pipeline - INFO - LangGraph Node [rerank_chunks]: 5 chunks passed score thresho
ld.
2026-09-02 02:03:39,444 - app.generation.llm - INFO - Trying Gemini model: 'gemini-3.5-flash'
2026-09-02 02:03:39,444 - google_genai.models - INFO - AFC is enabled with max remote calls: 10.
2026-09-02 02:03:39,445 - google_genai.models - WARNING - Direct use of automatic function calling (AFC) in Models.generat
e_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Model
s.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
2026-09-02 02:03:42,866 - httpx - INFO - HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemini
-3.5-flash:generateContent "HTTP/1.1 200 OK"
2026-09-02 02:03:42,874 - app.generation.llm - INFO - Gemini response received from model: 'gemini-3.5-flash'
2026-09-02 02:03:42,876 - app.pipeline.rag_pipeline - INFO - LangGraph Node [generate_answer]: Answer generated.
2026-09-02 02:03:42,879 - app.pipeline.rag_pipeline - INFO - LangGraph Node [validate_groundedness]: Status=HIGH
INFO: 127.0.0.1:59884 - "POST /api/v1/query HTTP/1.1" 200 OK

```

Fig 9.8: Backend log of a full pipeline execution — retrieval, rerank, Gemini generation, and groundedness validation stages

10. Testing and Results

10.1 Automated Unit & Integration Tests

The project includes a pytest-based test suite (tests/) covering the three most critical correctness layers of the system: API availability, the anti-hallucination groundedness logic, and the document ingestion pipeline.

Test File	Test Case	What It Verifies
tests/test_api.py	test_health_endpoint	GET /api/v1/health returns status “healthy” and vector count
tests/test_api.py	test_documents_endpoint	GET /api/v1/documents returns the indexed-documents list structure
tests/test_groundedness.py	test_groundedness_refusal	A correct refusal is classified as a grounded (N/A – Refusal) response
tests/test_groundedness.py	test_groundedness_high_confidence	A well-supported answer is scored HIGH with overlap > 0.5
tests/test_groundedness.py	test_groundedness_low_confidence	An unsupported / off-topic answer is correctly scored LOW
tests/test_ingestion.py	test_document_loader_txt	TXT files are loaded with correct filename, type, and page content
tests/test_ingestion.py	test_document_parser_clean	Whitespace and redundant newlines are normalised correctly
tests/test_ingestion.py	test_recursive_chunker	Long text is split into multiple chunks with correctly formatted chunk IDs

10.2 Functional / End-to-End Results

Beyond unit-level tests, the system was run live and exercised end-to-end using the sample document “AI Engineer Internship Task List.pdf”. The backend logs (Fig. 9.8) confirm a full pipeline execution: 5 raw chunks retrieved from FAISS, all 5 chunks kept by the reranker at the 0.25 threshold, a successful Gemini API call (model gemini-3.5-flash), and a final groundedness validation status of HIGH — all returned to the client with a 200 OK response from POST /api/v1/query.

The UI-level result for the query “Explain about advanced level” (Fig. 9.3) shows a HIGH-confidence grounded answer with 36 of 43 key answer terms independently verified against the retrieved context (84% overlap), along with five fully expandable source citations (Fig. 9.4, 9.5) each showing the exact filename, page number, chunk ID, similarity score, and excerpt — confirming that the citation and groundedness-reporting features work as designed.

Document lifecycle management was also verified live: Fig. 9.7 shows a DELETE /api/v1/documents/{filename} request correctly removing a previously indexed file (“Unit - V.pdf”, 75 chunks) and updating the remaining

vector count, confirming that the FAISS index rebuild logic functions correctly outside of the primary demo document.

11. Advantages

- Zero embedding cost: all vector embeddings are generated locally on CPU, with no per-call API fees or rate limits.
- Reduced hallucination risk through a two-layer defence: a strict grounding prompt at generation time, plus an independent post-hoc groundedness check.
- Transparent, verifiable answers: every response is traceable to an exact filename, page, and chunk, which builds user trust.
- Modular, decoupled architecture (ingestion, retrieval, generation, validation as separate modules) that is easy to test, extend, or swap components in.
- Resilient generation via a model fallback chain, reducing single-point-of-failure risk from one specific LLM model being unavailable.
- Reproducible deployment via Docker and docker-compose for both backend and frontend.

12. Limitations

- OCR is not supported: the system currently relies on text-extractable PDFs via pypdf, so scanned/image-only PDFs cannot be indexed.
- Retrieval uses dense vector search only; there is no hybrid keyword-based (e.g. BM25) search, which can occasionally miss exact-keyword or rare-term queries.
- The groundedness check is a heuristic, word-overlap-based metric rather than a semantic entailment model, so it can be imprecise for paraphrased answers.
- The system currently supports only PDF, TXT, and Markdown formats (e.g. DOCX or HTML are not yet supported).

There is no persistent conversational memory across queries — each question is handled independently.

13. Future Enhancements

- Integrate Tesseract OCR to support scanned/image-based PDF documents.
- Add hybrid search (dense vector + BM25 keyword search) for improved keyword-level precision.
- Replace the word-overlap groundedness heuristic with a semantic entailment or cross-encoder-based verification model.
- Extend supported formats to DOCX, HTML, and CSV documents.
- Add multi-turn conversational memory so follow-up questions can reference earlier context.
- Add user authentication and per-user document scoping for multi-tenant deployment.

14. Conclusion

DocuMind RAG Assistant demonstrates a complete, production-style approach to Retrieval-Augmented Generation that goes well beyond a basic LLM-wrapper chatbot. By combining local embeddings, a persistent FAISS vector index, relevance-based reranking, strictly grounded LLM prompting, and an independent groundedness validation layer, the system directly addresses the two core failure modes of naive RAG systems: irrelevant retrieval and ungrounded generation.

The working end-to-end results captured during development — successful multi-stage pipeline execution, HIGH-confidence grounded answers, and fully traceable citations — confirm that the system meets the objectives defined for the SHADOWFOX Advanced Level AI Engineer task: a modular, explainable, and practically engineered AI application rather than a simple demo. This internship provided valuable hands-on experience in designing multi-stage AI pipelines, working with vector databases, prompt engineering for factual grounding, and building decoupled, deployable AI services.

15. References

- FastAPI Documentation — <https://fastapi.tiangolo.com/>
- Streamlit Documentation — <https://docs.streamlit.io/>
- FAISS: A library for efficient similarity search — <https://github.com/facebookresearch/faiss>
- LangGraph Documentation — <https://langchain-ai.github.io/langgraph/>
- Sentence-Transformers Documentation — <https://www.sbert.net/>
- Google Gemini API Documentation — <https://ai.google.dev/>
- Docker Documentation — <https://docs.docker.com/>
- Project source code and README, DocuMind RAG Assistant (internal project repository).